

Section 3: Product Analytics Interview

This section covers the three workhorse frameworks of product analytics - funnels, retention, and cohorts - at the depth an interviewer expects from someone who has shipped funnel diagnosis to production. Throughout, I tie the concepts to Helios, where funnel logic lives in `fct_funnel` with monotonic `reached_*` flags, and retention/cohort math is computed deterministically by `stats-mcp.cohort_retention` under an explicit short-horizon caveat.

Funnels

Q1. What is the difference between step (step-to-step) conversion and overall conversion, and why does an analyst need both?

Ideal answer. Overall conversion is `purchasing_sessions / sessions` - the end-to-end yield of the whole funnel - while step conversion is the rate between two adjacent stages, e.g.

`add_to_cart_sessions / view_item_sessions`. Overall conversion tells you *how much* business you're losing; step conversion tells you *where* you're losing it, because the overall rate is the product of all step rates (`overall = step1 x step2 x ... x stepN`). I report both because a flat overall number can hide a single catastrophic step (a checkout that 90%-abandons), and conversely a single bad step rate is meaningless without knowing how many sessions actually reach it. The discipline is: watch overall as the headline KPI, then attribute movement to the specific step whose rate moved most. **Why Helios demonstrates it.** Helios materializes both `session_conversion_rate` (overall) and the four step rates (`view_to_cart_rate`, `cart_to_checkout_rate`, etc.) in the semantic layer, so the Decompose agent can decompose a movement in the headline rate down to the exact step that caused it before any hypothesis is formed. **Follow-ups.** If overall conversion is flat but two step rates moved in opposite directions, how do you surface that? / Why is multiplying step rates not exactly equal to overall conversion in real data? / How would you weight which step to investigate first?

Q2. Explain ordered vs unordered funnels. When is each appropriate?

Ideal answer. An ordered funnel requires users to complete steps in a specific sequence and counts a step only if the prior steps happened first (view -> cart -> checkout -> purchase); an unordered funnel counts whether each event occurred at all within the window, regardless of sequence. Ordered funnels model genuine sequential processes like e-commerce checkout where

order is causally meaningful, and they're stricter, so they report lower, more honest conversion. Unordered funnels suit feature-adoption questions ("did the user do A, B, and C in any order this week?") where sequence is irrelevant. The trap is using an unordered count for an inherently ordered process: you'll over-count "conversions" because a user who purchased via a deep link without viewing an item still satisfies an unordered "viewed AND purchased" filter. For a purchase funnel I default to ordered. **Why Helios demonstrates it.** Helios's macro funnel (`session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`) is strictly ordered and enforced by the `reached_*` max-downstream flags, so a session marked `reached_begin_checkout` provably passed through the earlier stages. **Follow-ups.** How do you handle a user who skips a step (e.g. one-click buy)? / What breaks if you compute an ordered funnel without a per-user timestamp ordering? / When would you deliberately relax ordering?

Q3. How do you choose the conversion window for a funnel, and what bias does the choice introduce?

Ideal answer. The window is the time allowed for a user to complete downstream steps after entering, and it should match the natural decision latency of the action: minutes-to-hours for an impulse e-commerce purchase, days-to-weeks for a high-consideration B2B signup. Too short a window under-counts conversions that legitimately complete later (truncation bias); too long inflates conversion by crediting unrelated later sessions and makes the funnel slow to react to recent changes. The right approach is to inspect the empirical distribution of time-to-convert and set the window at a high percentile (e.g. p90) of genuine conversions, then hold it fixed so trends are comparable. I also distinguish *session-scoped* funnels (window = one session) from *user-scoped* funnels (window spans sessions), since they answer different questions. **Why Helios demonstrates it.** Helios scopes its funnel to a single session - session key (`user_pseudo_id`, `ga_session_id`) - which makes the conversion window unambiguous (intra-session) and sidesteps cross-session attribution noise, a deliberate simplification stated as a known limitation rather than hidden. **Follow-ups.** How would you measure time-to-convert to justify the window? / What changes if you move from session-scoped to 7-day user-scoped? / How does window choice interact with seasonality?

Q4. A funnel step's drop-off jumped last week. Walk me through how you attribute the cause.

Ideal answer. First I confirm the drop is real and not noise - is the change statistically significant given the sample reaching that step, or is it a small-N artifact? Then I decompose the rate change: did the *segments' behavior* worsen (rate-change), or did the *traffic mix* shift toward inherently lower-converting segments (mix-shift)? These have completely different fixes - rate-change points at a product/UX/pricing defect, mix-shift points at acquisition. I then segment the step by device, channel, geo, and landing page to localize the regression, and cross-check against deploy logs,

tracking changes, and known launches before concluding. Finally I quantify the impact in dollars (sessions lost x downstream value) so the finding is prioritized by money, not by percentage-point drama. **Why Helios demonstrates it.** This is Helios's core loop: `stats-mcp.significance_test` gates the anomaly, `stats-mcp.decompose_change` splits the movement into `mix_effect` / `rate_effect` / `interaction`, and the Diagnose agent drills the rate effect first because that's where real behavior change lives. **Follow-ups.** What's your significance test for a proportion change? / How do you rule out a tracking/data-quality regression? / How do you convert a step drop into revenue-at-risk?

Q5. What is mix-shift vs rate-change in a funnel, and why does conflating them cause wrong decisions?

Ideal answer. For an aggregate rate $R = \sum_i (w_i * r_i)$ where w_i is a segment's volume share and r_i its conversion rate, a change decomposes as $mix = \sum(dw_i * r_i(t_0))$, $rate = \sum(w_i(t_0) * dr_i)$, and $interaction = \sum(dw_i * dr_i)$. Mix-shift means the aggregate moved purely because traffic composition changed (e.g. a paid-social campaign flooded the top of funnel with low-intent mobile users) even though *every segment's* conversion held steady; rate-change means in-segment behavior genuinely changed. Conflating them is the classic error: you "fix" a checkout you think is broken when actually the checkout is fine and acquisition got cheaper-and-worse. Decomposing first tells you whether to call product or marketing. **Why Helios demonstrates it.** This decomposition identity is Helios's technical centerpiece - `stats-mcp.decompose_change` computes it deterministically in Python (never in token-space), which is exactly how Helios dissolves Simpson's paradox before forming any hypothesis. **Follow-ups.** Give an example where overall conversion rises but every segment's rate falls. / Why compute this in real code rather than letting an LLM estimate it? / How big does interaction have to be before you report it separately?

Q6. Why can an aggregate funnel mislead you (Simpson's paradox), and how do you guard against it?

Ideal answer. Simpson's paradox is when a trend present in every subgroup reverses or vanishes in the pooled data because the subgroup *weights* changed. In funnels this means overall conversion can rise while mobile, desktop, and tablet conversion each fall - simply because desktop (the higher-converting segment) gained share. The aggregate is a weighted average, and weighted averages move for two reasons: the weights or the rates. The guard is to never trust a pooled rate movement on its own: always decompose into mix vs rate, and always compute rates as `SUM(numerator)/SUM(denominator)` after grouping rather than averaging per-segment ratios, which silently re-weights every segment to equal. **Why Helios demonstrates it.** Helios bakes this in two places: the dbt convention that rates are `SUM(num)/SUM(den)` post-group (never a mean of ratios), and `decompose_change` as a mandatory step so no finding ships on an un-

decomposed aggregate. **Follow-ups.** Why does averaging per-segment ratios re-weight the data? / Which dimension would you decompose by first and why? / How do you present a Simpson reversal to a non-technical exec?

Q7. How do you analyze a funnel by segment without drowning in dimension combinations?

Ideal answer. I treat segmentation as a guided search, not a full cross-join. Start with the single dimension whose decomposition contributes the largest *rate effect* to the movement, drill into its worst-performing level, then conditionally expand to a second dimension only within that branch - a best-first traversal rather than enumerating device x channel x geo x landing-page exhaustively. I cap segment cardinality and enforce a minimum sample per cell so I'm not chasing 12-session segments with noisy rates. The output is a short, ranked path ("Paid Social -> mobile -> /sale landing page") rather than a 400-row pivot, because a prioritized narrative is what drives action. **Why Helios demonstrates it.** The Diagnose agent runs a best-first hypothesis-tree RCA over Helios's conforming dimensions (`device_category` , `channel_group` , `country` , etc.), expanding only the most promising branch and SQL-verifying each node, which keeps it inside the 5 GiB byte budget instead of blowing it on a full cube. **Follow-ups.** What minimum sample do you require per segment cell? / How do you stop the tree from over-fitting to one noisy segment? / How do you handle high-cardinality dimensions like `landing_page`?

Q8. How would you design and QA a funnel data model so step rates are never greater than 1 and never silently wrong?

Ideal answer. The invariant is monotonicity: `sessions ≥ reached_step1 ≥ reached_step2 ≥ ... ≥ reached_purchase` . I enforce it by defining each step flag as "reached this step *or any later step*" (a max-downstream flag) computed from the maximum stage achieved per session, rather than independent per-event booleans that can produce a later step with no earlier one. Then every step rate is guaranteed ≤ 1 by construction. I QA it with tests asserting the monotonic chain and with a reconciliation that the funnel's terminal count matches the orders fact to the cent. These transforms fail *silently* - a subtly wrong sessionization poisons every downstream number - so they get golden-value tests, not just spot checks. **Why Helios demonstrates it.** Helios's `int_ga4__funnel_steps` uses `reached_*` monotonic flags (the `did_*` naming is retired precisely because it allowed non-monotonic states), and revenue at the terminal step reconciles against `fct_orders` via the `revenue_reconciles` test - the keystones flagged as silent-failure risks. **Follow-ups.** Show the SQL shape of a max-downstream flag. / What test catches a sessionization bug before it reaches the funnel? / Why is reconciliation to $\leq 0.5\%$ drift part of the funnel's definition of done?

Retention

Q9. Define classic (N-day), rolling, and bracket (range) retention. When do you use each?

Ideal answer. Classic N-day retention measures the share of a cohort active *exactly* on day N after acquisition - precise but harsh and spiky for low-frequency products. Rolling (a.k.a. "N-day or later" / unbounded) retention counts users active on day N *or any day after*, which is smoother and better for products used irregularly. Bracket/range retention buckets activity into windows (day 0, days 1-7, days 8-30) and counts a user retained if active anywhere in the bracket - the practical default for weekly or monthly products because it tolerates natural usage cadence. The choice must match usage frequency: applying classic day-7 retention to a product people use monthly will report near-zero retention that's an artifact of the metric, not the product. **Why Helios demonstrates it.** Helios exposes retention through `stats-mcp.cohort_retention` as a deterministic, parameterized computation, so the retention *definition* (classic vs rolling vs bracket) is an explicit, auditable argument rather than an ad-hoc SQL decision buried in a query. **Follow-ups.** Which would you pick for a monthly-use product and why? / How does rolling retention relate to a survival/Kaplan-Meier curve? / What's the failure mode of classic retention for low-frequency apps?

Q10. What does a retention curve look like, and what do its shape, slope, and floor tell you?

Ideal answer. A retention curve plots the fraction of a cohort still active against days/weeks since acquisition. Healthy products show a steep initial drop (the "shedding" of low-intent signups) that then *flattens into a non-zero asymptote* - the "smile" or plateau - which is the retained core that drives long-term value. A curve that decays to zero with no floor means no durable habit formed and the product is leaky no matter how much you spend on acquisition. The two things I read are the *floor* (does it asymptote above zero?) and the *slope after the elbow* (a slowly declining plateau still erodes the base). A rising or "smiling" curve, where later periods recover, signals genuine product-market fit. **Why Helios demonstrates it.** Helios computes the per-period retained fractions via `cohort_retention` so the curve's elbow and floor are quantified outputs, but it explicitly caps interpretation at the ~3-month data window - it will not extrapolate a plateau it cannot observe. **Follow-ups.** How do you distinguish a true plateau from not-enough-data? / What slope on the plateau still worries you? / How would you compare two cohorts' curves statistically?

Q11. N-day vs unbounded retention: what's the trade-off and which is more honest?

Ideal answer. N-day (exact-day) retention is unbiased for the precise question "active on day N" but is high-variance and punishes any product not used daily, often understating real engagement. Unbounded ("day N or later") retention is smoother and arguably more honest about whether a user is *still a user*, but it's monotonically non-increasing only in N and can mask declining frequency because a user who logs in once on day 60 counts as retained through day 30. Neither is universally "right" - the honest move is to state the definition explicitly and pick the one whose question matches the decision. For churn/health monitoring I lean unbounded; for

measuring a specific day-1 onboarding intervention I use exact N-day so the signal isn't diluted by later activity. **Why Helios demonstrates it.** Because retention is a tool argument in `stats-mcp.cohort_retention`, Helios forces the definition to be named and stored with the diagnosis in memory, so a later run reads back retention computed the same way - no silent redefinition between runs. **Follow-ups.** Why is unbounded retention monotonic in N? / When does unbounded retention hide a real decline? / Which do you put on the exec dashboard?

Q12. Why is retention often called the truest signal of product health - truer than acquisition or even revenue?

Ideal answer. Acquisition and top-line revenue can be bought - a spend spike inflates both without any improvement in the product - whereas retention measures whether users came *back* unprompted, which is the closest observable proxy for delivered value and the engine of compounding growth. Retention sets the ceiling on LTV (LTV is essentially the integral under the retention curve times per-period value), so a leaky bucket caps every other metric no matter how good the funnel is. It's also lagging and hard to game, which is exactly what makes it trustworthy. The mature framing: funnels tell you if you can *get* users, retention tells you if you *deserve* them.

Why Helios demonstrates it. Helios prices movements in dollars of revenue-at-risk and prescribes experiments, but it's explicit that on a ~3-month observational window it reports *short-horizon retention proxies with stated assumptions* rather than claiming true LTV - treating retention as the health signal while being honest about how far it can see. **Follow-ups.** How is LTV related to the area under the retention curve? / Why is retention harder to game than revenue? / What short-horizon proxy would you trust on 3 months of data?

Q13. What is survivorship bias in retention analysis and how does it distort conclusions?

Ideal answer. Survivorship bias is drawing conclusions from only the users who "survived" to the point of measurement while ignoring those who churned earlier - so the surviving population looks healthier and more engaged than the original cohort ever was. It shows up when you compute "average engagement of active users" over time: the average rises not because anyone improved but because the disengaged dropped out, leaving a denominator of self-selected loyalists. Another form is analyzing only cohorts old enough to have "matured," which excludes recently-acquired (and possibly worse-retaining) users. The fix is to always anchor metrics to the *original cohort denominator at acquisition*, and to be explicit when a metric conditions on still being active. **Why Helios demonstrates it.** `cohort_retention` always divides by the cohort's *acquisition-time* size, not the surviving count, so retained fractions can't drift upward purely from attrition - and the Critic agent flags "insufficient sample" / survivorship-style confounds as a refutation category before a finding ships. **Follow-ups.** Where does "average sessions per active user" hide survivorship? / How do you correct an engagement metric for it? / How does the Critic catch this class of error?

Q14. How would you statistically compare retention between two cohorts (e.g. before vs after a change)?

Ideal answer. I compare retention at matched periods-since-acquisition (day-7 vs day-7), not calendar dates, and test the difference in retained proportions with a two-proportion z-test or chi-square, reporting a confidence interval on the gap, not just a point delta. For the whole curve I'd use a log-rank test on the survival curves so I'm comparing shapes rather than one cherry-picked day. Crucially I check that the cohorts are comparable on mix (channel, device, geo) first - a "retention improvement" that's really a mix-shift toward better-retaining channels is the same Simpson trap as in funnels. Because this is observational data, I frame it as a quasi-experimental pre/post or difference-in-differences readback, not a causal A/B claim. **Why Helios demonstrates it.** Helios routes all such tests through `stats-mcp.significance_test` (deterministic, seeded) and is explicit that on its historical dataset it runs quasi-experimental pre/post and diff-in-diff readbacks rather than live A/Bs - sizing experiments it cannot actually run live. **Follow-ups.** Why match on period-since-acquisition rather than calendar week? / When is a log-rank test better than a per-day z-test? / How does diff-in-differences strengthen a pre/post retention claim?

Q15. A retention curve flattened into a plateau after a release. Is that good, and how do you verify it's real?

Ideal answer. A plateau above zero is the goal - it signals a retained core - but after a release I'm suspicious of three alternatives before I celebrate: (1) not enough elapsed time, so the "plateau" is just the youngest cohorts not having had time to churn yet; (2) survivorship, where the denominator quietly shifted; and (3) seasonality, where a holiday lull or spike coincided with the release. I verify by comparing same-age cohorts across the release boundary, checking significance on the plateau-period retention, and decomposing whether the improvement is in-segment (real) or a mix-shift toward stickier channels. Only after those refutations survive do I attribute the plateau to the release. **Why Helios demonstrates it.** This maps directly onto the Critic's adversarial refutation categories - seasonality decoy, insufficient sample, mix-confound, data-quality - which must all return PASS before Helios reports the plateau as a genuine win rather than a DOWNGRADE/DROP. **Follow-ups.** How do you tell a real plateau from cohort immaturity? / What seasonality control would you use on GA4 holiday data? / Why decompose a retention gain by segment?

Q16. How do you handle low-frequency products where day-N retention looks terrible by construction?

Ideal answer. For a product genuinely used weekly or monthly, daily retention will look near-zero even with perfect engagement, so I switch the *unit of time* to match cadence - weekly or monthly retention buckets (bracket retention) - and define "active" by a meaningful action, not just a session open. I also look at frequency/recency directly (how often does the core return, and how

recently) rather than forcing a daily lens. The mistake is keeping the daily metric and concluding the product is failing; the metric is failing the product. Picking the retention granularity is itself an analytical decision that should be justified from the observed usage-interval distribution. **Why Helios demonstrates it.** Because retention granularity is a parameter to `stats-mcp.cohort_retention`, Helios can switch between daily/weekly/monthly bucketing without rewriting SQL - and it pairs retention with RFM-style recency/frequency via `stats-mcp.rfm_segment` for low-frequency populations. **Follow-ups.** How do you pick weekly vs monthly from the data? / What defines "active" for a low-frequency product? / How does RFM complement a retention curve here?

Q17. What's the relationship between retention, churn, and engagement, and how do you avoid double-counting them as separate wins?

Ideal answer. Retention and churn are complements at a fixed horizon (period churn = 1 - period retention), so reporting both a "retention up" and a "churn down" headline from the same data is double-counting the same movement. Engagement (depth/frequency of use among the active) is a *different* axis: you can have stable retention with rising engagement, or shrinking-but-deepening usage. The clean framing separates the *width* of the base (retention/churn) from the *intensity* of use (engagement), and recognizes that engagement often *predicts* future retention - so I use engagement as a leading indicator and retention as the confirming lagging metric, never as two independent KPIs to celebrate twice. **Why Helios demonstrates it.** Helios keeps these distinct in its semantic layer - `engagement_rate` / `engaged_sessions` measure intensity while `cohort_retention` measures the durable base - so a Decision Brief never books a single underlying movement as two separate findings. **Follow-ups.** Give a case where retention is flat but engagement rises - what do you do? / Why is engagement a leading indicator of retention? / How would you test whether engagement predicts next-period retention?

Cohorts

Q18. What is a cohort, and what's the difference between an acquisition cohort and a behavioral cohort?

Ideal answer. A cohort is a group of users sharing a defining event in a common time window, analyzed together over their lifetime so you compare like-aged users rather than mixing tenures. An *acquisition* cohort is grouped by *when* they arrived (the January-signups, the week-of-launch users) and answers "is each successive vintage of users better or worse?" A *behavioral* cohort is grouped by *what they did* (users who completed onboarding, users who added a payment method) and answers "does doing X change downstream retention/revenue?" Acquisition cohorts isolate temporal/quality trends in the funnel top; behavioral cohorts isolate the value of specific actions and are how you find activation "aha" moments. The two answer different questions and shouldn't be conflated. **Why Helios demonstrates it.** Helios's data model carries `is_new_user`

and `session_number_bucket` dimensions plus an acquisition-date grain, so `cohort_retention` can be sliced as an acquisition cohort (by arrival week) or a behavioral one (e.g. sessions that `reached_add_to_cart`) without leaving governed SQL. **Follow-ups.** Which cohort type finds an activation "aha" moment? / How would you define a behavioral cohort in this GA4 funnel? / What confound threatens a behavioral-cohort conclusion?

Q19. How do you read a cohort retention triangle/grid, and what patterns matter?

Ideal answer. A cohort grid has one row per acquisition cohort and one column per period-since-acquisition, with the retained fraction in each cell - so you read *down a column* to compare the same age across cohorts (is the day-7 number improving for newer vintages?) and *across a row* to trace one cohort's decay curve. Verticals isolate whether product/onboarding changes lifted later cohorts (cohort-over-cohort improvement at fixed age); diagonals reveal calendar-time shocks (a holiday or outage hitting all live cohorts at once). The single most important read is the column trend at a stable horizon, because that's the apples-to-apples signal that the product is getting stickier independent of how much time has elapsed. **Why Helios demonstrates it.** Helios's `cohort_retention` output is exactly this period-since-acquisition grid, and the Decompose/Diagnose agents read columns (same-age cohort-over-cohort) so an improvement isn't faked by simply having older, more-decayed cohorts in the mix. **Follow-ups.** What does a diagonal pattern tell you vs a vertical one? / Why compare at fixed age rather than fixed calendar date? / How do you spot a tracking change in the grid?

Q20. What does "reading cohort-over-cohort" mean and what's the most common mistake?

Ideal answer. Reading cohort-over-cohort means comparing successive acquisition cohorts at the *same maturity* - January's day-30 retention vs February's day-30 retention - to judge whether the product is improving for each new vintage. The most common mistake is comparing cohorts at the same *calendar date* instead of the same age: a newer cohort has had less time to churn, so it will always look artificially better, an "immaturity" illusion that evaporates as they age. The second mistake is comparing cohorts of wildly different *composition* (a paid-campaign cohort vs an organic cohort) and attributing a retention gap to the product when it's really an acquisition-mix difference - the same mix-vs-rate problem that haunts funnels. **Why Helios demonstrates it.** Helios anchors cohort comparisons to period-since-acquisition (not calendar) and runs `decompose_change` to separate whether a cohort-over-cohort shift is mix (cohorts sourced differently) or rate (genuinely better retention) before crediting a release. **Follow-ups.** How does cohort immaturity inflate a young cohort? / How do you control for acquisition-mix differences between cohorts? / What's the minimum elapsed time before a cohort comparison is fair?

Q21. Why is the ~3-month data window a serious caveat for cohort and retention conclusions, and how do you handle it honestly?

Ideal answer. With only ~3 months of data, the earliest cohort has at most ~90 days of observable history and the latest has almost none, so any long-horizon retention or LTV claim is extrapolation beyond what the data can support - the curve's true asymptote and tail are simply unobserved. The honest handling is to (1) report only short-horizon proxies (e.g. day-7, day-30 retention) and label them as such, (2) state the assumptions any projection rests on rather than presenting a smooth LTV number, and (3) restrict cohort-over-cohort comparisons to ages every compared cohort has actually reached. Pretending to a 12-month LTV from 3 months of data is exactly the over-claiming that erodes trust with a senior stakeholder. **Why Helios demonstrates it.** This is one of Helios's explicit maturity points: the dataset is observational and ~3 months (Nov 2020-Jan 2021), so Helios *defers true LTV*, reports short-horizon retention proxies with stated assumptions, and won't extrapolate a tail it can't see. **Follow-ups.** What horizon would you trust on this dataset and why? / How would you caveat an LTV projection to an exec? / How does limited history interact with cohort immaturity?

Q22. How would you build an acquisition-cohort retention analysis on GA4 data, given user_id is mostly null?

Ideal answer. I'd grain the cohort on the first-seen date per identity, but I have to be explicit that GA4's `user_id` is almost always null, so identity is cookie-grain (`user_pseudo_id`) - meaning these are *cookie-based* cohorts that over-count users who clear cookies or switch devices and can't stitch cross-device journeys. I assign each `user_pseudo_id` to its acquisition week, then compute retained fractions at each period-since-acquisition, comparing cohorts down the column. I'd state the cookie-grain assumption up front because it caps the precision of any user-level retention claim, and I'd avoid implying these are true deduplicated humans. Naming the limitation is what makes the analysis credible. **Why Helios demonstrates it.** Helios is explicit that `user_id` is almost always null so identity is cookie-grain (`user_pseudo_id`) and user-level metrics are cookie-based approximations - the cohort math in `cohort_retention` inherits and surfaces that caveat rather than hiding it. **Follow-ups.** How does cookie-grain identity bias retention up or down? / What would change with stitched cross-device identity? / How do you communicate "cookie-based, not user-based" to a PM?

Q23. How do you decide cohort granularity (daily vs weekly vs monthly) and cohort size?

Ideal answer. Granularity trades resolution against statistical stability: daily cohorts give fine temporal detail but small, noisy cells; monthly cohorts are stable but blur fast-moving changes. I pick the grain from two things - the product's natural usage cadence and the per-cohort sample size needed for the retained-fraction confidence intervals to be tight enough to act on. On a short window with moderate traffic, weekly cohorts are usually the sweet spot: enough users per cohort to be significant, enough cohorts to see a trend. I also align cohort width to the retention bucket width so I'm not comparing daily cohorts on monthly retention or vice versa, which mismatches

the two time axes. **Why Helios demonstrates it.** Helios carries both `day` and `week` dimensions, and because cohort grain and retention bucketing are parameters to `cohort_retention`, it sizes cohorts to keep per-cell samples above the threshold the `significance_test` needs - all under the fixed 5 GiB byte budget. **Follow-ups.** How does cohort size affect the confidence interval on retention? / Why align cohort width with retention-bucket width? / What sample floor do you set per cohort cell?

Q24. How would you find an "aha moment" / activation metric using behavioral cohorts, and what's the causal caveat?

Ideal answer. The standard play is to build behavioral cohorts on candidate early actions (added an item, completed checkout setup, returned within 24h) and compare downstream retention between users who did vs didn't do each action within their first session/week - the action with the largest, most stable retention lift is the activation candidate. The non-negotiable caveat is that this is *correlational*: motivated users both perform the action and retain, so the action may be a marker of intent rather than a cause. To move toward causation I'd design an experiment that nudges the action for a random subset, or at minimum run a quasi-experimental matched comparison and stress-test for the obvious confounds before recommending the team optimize for it. **Why Helios demonstrates it.** Helios's Prescribe agent turns such a candidate into a *powered* experiment card (via `experiment-mcp.power_analysis / design_experiment`) precisely because Helios refuses to present a behavioral-cohort correlation as a causal lever without sizing the experiment to confirm it. **Follow-ups.** Why is a behavioral-cohort retention lift not proof of causation? / How would you design an experiment to confirm the aha moment? / What confounds would the Critic raise here?

Q25. Tie it together: how do funnels, retention, and cohorts combine into a single growth diagnosis?

Ideal answer. They're three lenses on one system and most powerful when chained. The funnel localizes *where* in the session conversion breaks and prices the leak in dollars; the mix-vs-rate decomposition says whether to fix product or acquisition; cohorts then ask whether the leak is a one-time shock (a diagonal calendar event) or a structural decline in successive vintages (a worsening column); and retention answers whether the users you *do* convert actually stick - because fixing a funnel that feeds a leaky bucket just spends faster. The senior synthesis is a single prioritized narrative: the largest dollar-impact, statistically-significant, mix-vs-rate-disambiguated finding, with the cohort/retention context that says whether it's acute or chronic, plus a prescribed, powered experiment - all bounded by an honest statement of what the 3-month window can and can't support. **Why Helios demonstrates it.** This is precisely Helios's end-to-end pipeline: Monitor -> Decompose (mix/rate) -> Diagnose (funnel/segment RCA) -> Critic (refute) -> Prescribe (powered card) -> Narrator (Decision Brief), with funnel logic in `fct_funnel / reached_*`, cohort/retention math in `stats-mcp`, and the whole chain graded

against the offline 85%-vs-45% benchmark. **Follow-ups.** Which lens do you reach for first on a revenue drop and why? / How do you keep three analyses from producing three disjoint findings? / How does the dollar-impact ranking arbitrate between a funnel fix and a retention fix?